02. Getting Started with Sorting Problems

Hyunjong Choi, Ph.D.

Assistant Professor of Computer Science

San Diego State University

Outline

- Background
  - Pseudocode
  - Mathematical induction
  - Fundamentals of algorithms

- Insertion sort (Textbook 3rd edition 16 – 28)

- Merge sort (Textbook 3rd edition 29 – 37)

An alternative approach to sorting problems

- **Divide-and-conquer** approach: break the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems *recursively*, and then combine these solutions to create a solution to the original problem

- The divide-and-conquer paradigm involves three steps:
  - **Divide** the problem into a number of subproblems that are smaller instances of the same problem.
  - **Conquer** the subproblems by solving them recursively (further divide). If the subproblem size is small enough, just solve it in a straightforward manner.
  - **Combine** the solutions to the subproblems into the solution for the original problem.

Ideas of merge sort

Divide

- Divide the n-element sequence into *two* subsequences
- Each subsequence has a size of $n/2$ elements

Conquer

- Sort the two subsequences *recursively* using merge-sort
- The recursion stops when it reaches the "bottom", which has only 1 element because there is no work to be done

Combine

- Merge the ***two sorted*** subsequences to produce the sorted answer
- The key step that is the ***merging*** of two sorted sequences

Overview of merge sort implementation

Design "**MERGE**" procedure as a part of "MERGE-SORT" ⇨Combine step

Design "**MERGE-SORT**" using recursive algorithm and utilize "MERGE" procedure

MERGE-SORT(A, p, r)

```
1.  if p < r
2.      q = ⌊(p + r)/2⌋
3.      MERGE-SORT(A, p, q)
4.      MERGE-SORT(A, q+1, r)
5.      MERGE(A, p, q, r)
```

MERGE(A, p, q, r)

```
1.  n1 = q − p + 1
2.  n2 = r − q
3.  let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4.  for i = 1 to n1
5.      L[i] = A[p + i − 1]
6.  for j = 1 to n2
7.      R[j] = A[q + j]
8.  L[n1 + 1] = ∞
9.  R[n2 + 1] = ∞
10. i = 1
11. j = 1
12. for k = p to r
13.     if L[i] ≤ R[j]
14.         A[k] = L[i]
15.         i = i + 1
16.     else A[k] = R[j]
17.         j = j + 1
```

Design the MERGE procedure

- A card-playing analogy:
  - Assumptions: Two sorted piles of cards as input, with smallest cards on top
  - Goal: Wish to merge the two piles into a single sorted output pile
  - *Strategy*: ***choose the smaller of the two cards on top of the two input piles***, and place it onto the output pile
  - Repeat this step until one input pile is empty
  - Take the remaining input pile and place it onto the output pile

- Each basic step takes constant time (just comparing two cards).
- Since we perform at most n basic steps, merging takes $\boldsymbol{n}$ **(linear)** time.

Pseudocode of MERGE

```
MERGE(A, p, q, r)
1.   n1 = q − p + 1
2.   n2 = r − q
3.   let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4.   for i = 1 to n1
5.        L[i] = A[p + i − 1]
6.   for j = 1 to n2
7.        R[j] = A[q + j]
8.   L[n1 + 1] = ∞
9.   R[n2 + 1] = ∞
10.  i = 1
11.  j = 1
12.  for k = p to r
13.       if L[i] ≤ R[j]
14.            A[k] = L[i]
15.            i = i + 1
16.       else A[k] = R[j]
17.            j = j + 1
```

A is an array and p, q, and r are indices, such that $p \le q < r$

Computes the length n_1 of the subarray $A[p..q]$

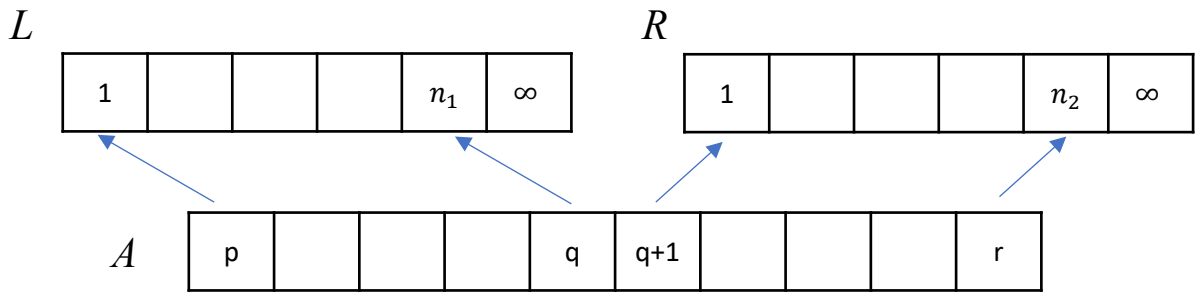
Computes the length n_2 of the subarray $A[q+1..r]$

Create two new arrays to store the two subarrays, of length $n_1 + 1$ and $n_2 + 1$ respectively

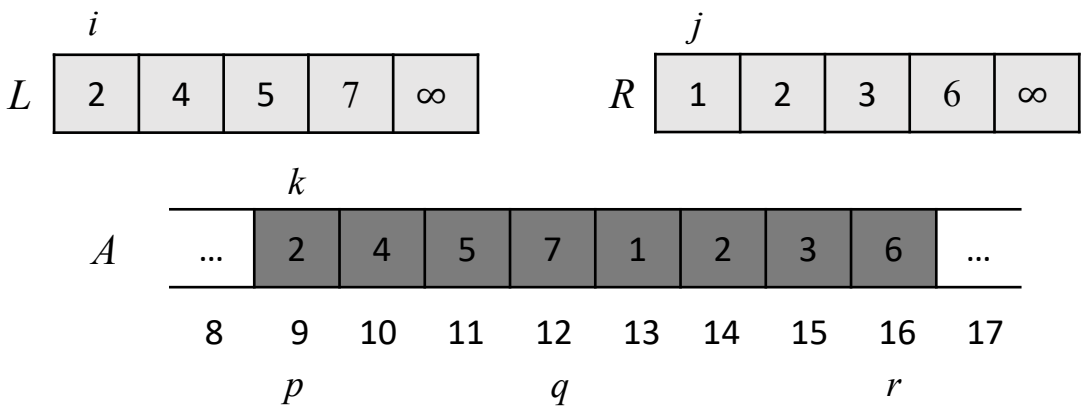
Copy the original elements in A to L and R

At the end of each array, place a **sentinel** value ∞, which is larger than any other value.
To avoid the operation of checking whether either array is empty in each basic step

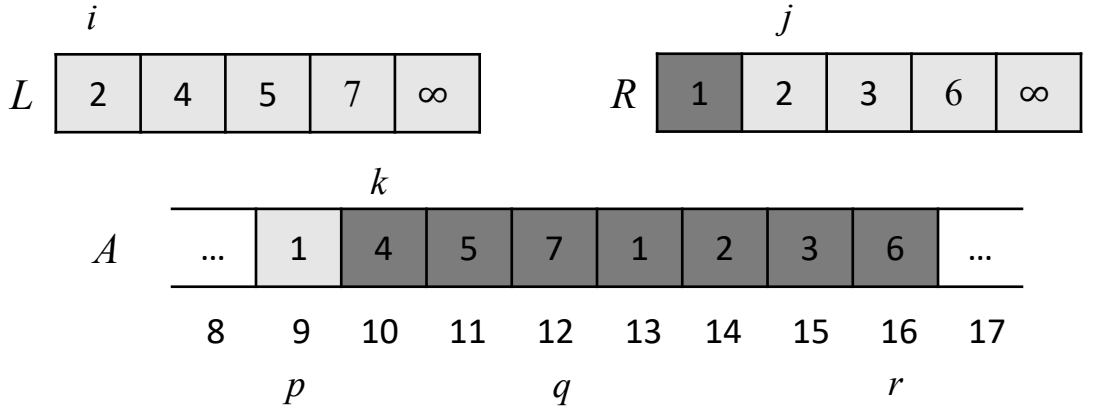
Perform $r - p + 1$ basic steps of comparison and copy



Example of basic merging steps

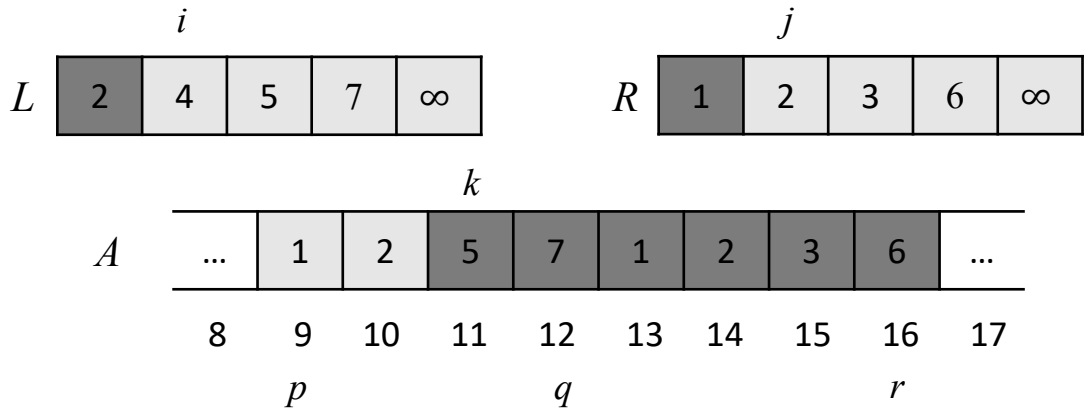


(a)



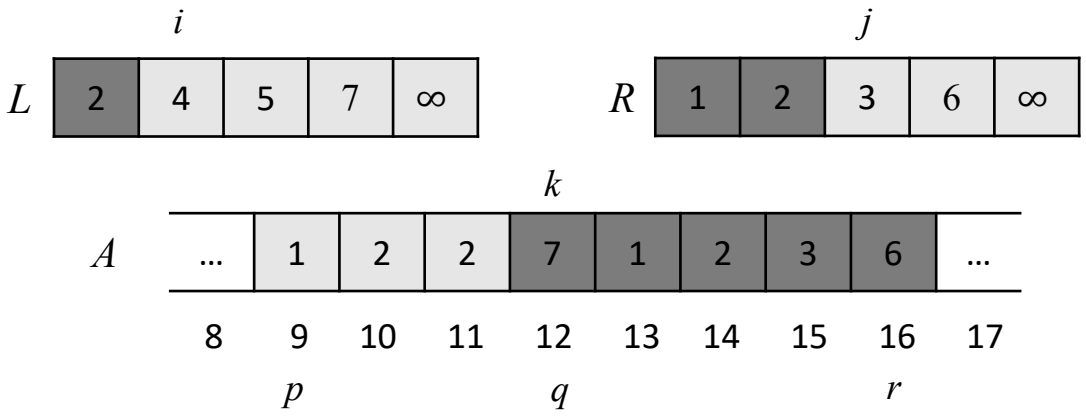
(b)

```
10.  i = 1
11.  j = 1
12.  for k = p to r
13.       if L[i] ≤ R[j]
14.            A[k] = L[i]
15.            i = i + 1
16.       else A[k] = R[j]
17.            j = j + 1
```

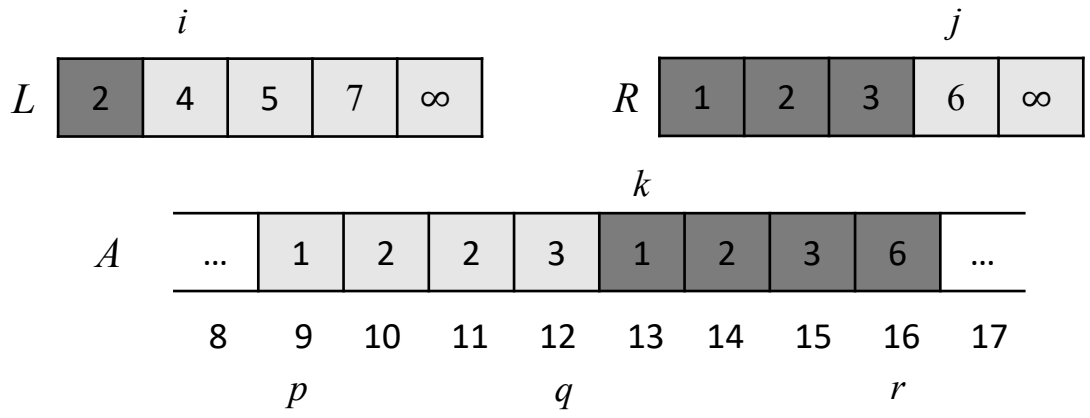


(c)

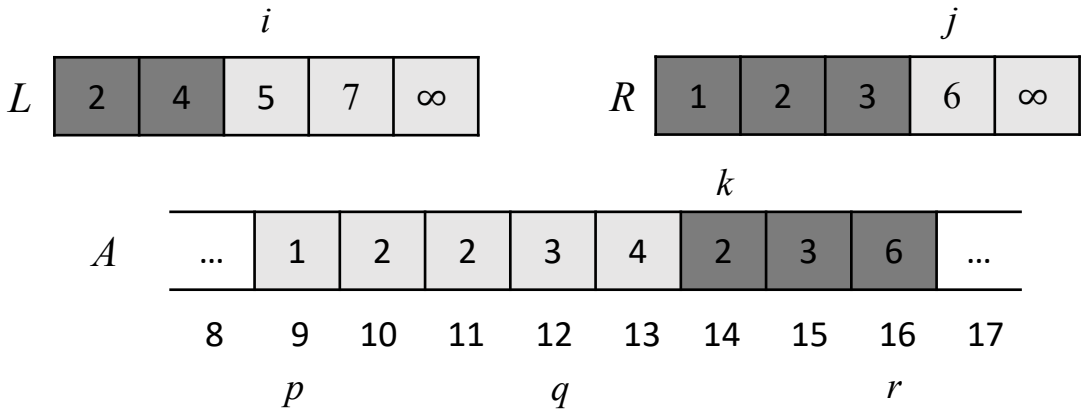
Example of basic merging steps (cont.)



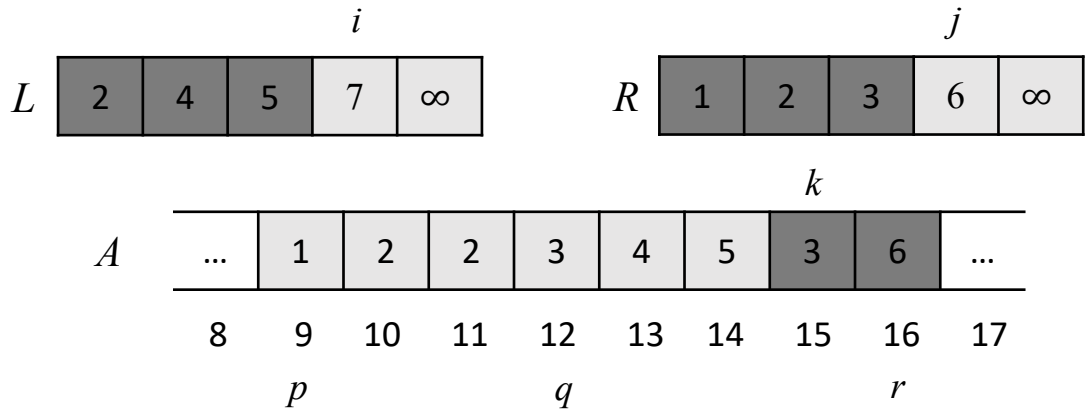
(d)



(e)

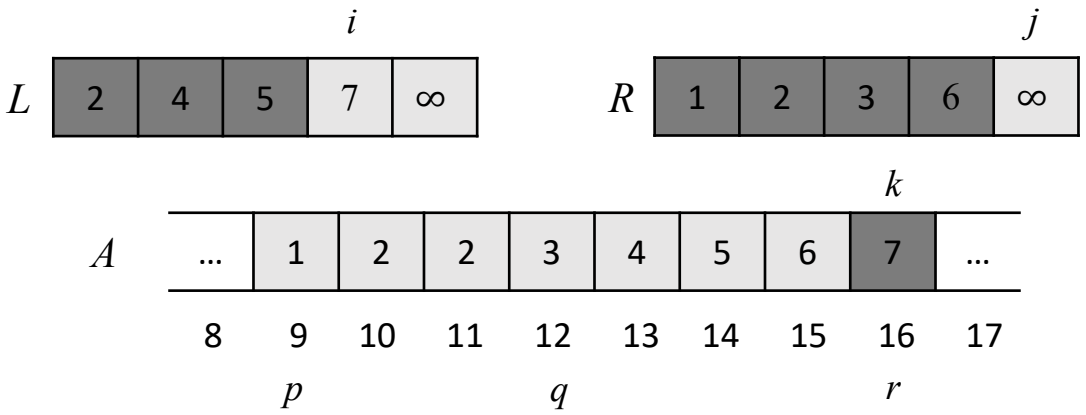


(f)

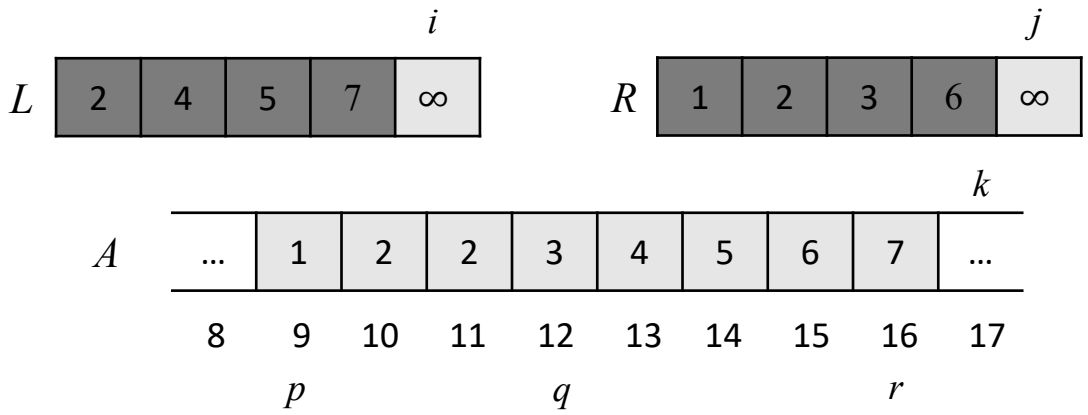


(g)

Example of basic merging steps (cont.)



(h)



(i)

At this point, $A[9..16]$ is sorted

Optional

Check loop invariant (optional)

- **Loop invariant**: "at the start of each iteration, the subarray $A[p: k-1]$ contains the $k-p$ smallest elements of $L[1: n_1+1]$ and $R[1: n_2+1]$, in sorted order. $L[i]$ and $R[j]$ are the smallest elements of their arrays that have not been copied back into A."

```
12.  for k = p to r
13.      if L[i] ≤ R[j]
14.          A[k] = L[i]
15.          i = i + 1
16.      else A[k] = R[j]
17.          j = j + 1
```

- ***Initialization***: prior to the first iteration, $k = p$
  - $A[p: k-1]$ is empty, which contains 0 smallest elements from L and R.
  - Since $i = j = 1$, $L[i]$ and $R[j]$ are the smallest elements.
- ***Maintenance***: each iteration of the loop maintains the invariant
  - Suppose that $L[i] \leq R[j]$, then $L[i]$ is the smallest element not yet copied back.
  - Because $A[p: k-1]$ contains $k-p$ smallest elements, after Line 14 (copying back), the subarray $A[p: k]$ will contain $k-p+1$ smallest elements
  - Incrementing k (done by **for** loop) and i (Line 15) reestablishes the loop invariant for the next iteration
  - If $L[i] > R[j]$, same applies.
- ***Termination***:
  - At termination, $k = r+1$. Thus, the subarray $A[p: k-1]$, which is $A[p: r]$ contains $r-p+1$ smallest elements
  - L and R together contain $r-p+3$ elements. All but the two largest sentinel values have been copied back.

Running time of merge procedure

- MERGE procedure takes time $\Theta(n)$, where $n = r - p + 1$ is the total number of elements being merged.

```
MERGE(A, p, q, r)
1.   n1 = q – p + 1
2.   n2 = r – q
3.   let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4.   for i = 1 to n1
5.        L[i] = A[p + i – 1]
6.   for j = 1 to n2
7.        R[j] = A[q + j]
8.   L[n1 + 1] = ∞
9.   R[n2 + 1] = ∞
10.  i = 1
11.  j = 1
12.  for k = p to r
13.       if L[i] ≤ R[j]
14.            A[k] = L[i]
15.            i = i + 1
16.       else A[k] = R[j]
17.            j = j + 1
```

Line 1 – 3 takes constant time

Θ notation

Line 4 – 7 takes $\Theta(n_1 + n_2) = \Theta(n)$ time

Line 8 – 11 takes constant time

Line 12 – 17 have n iterations, each of which takes $\Theta(n)$ time

Together, takes $\Theta(n)$ time

Implement merge sort

- We can use the MERGE procedure as a subroutine to implement the MERGE-SORT procedure

```
MERGE-SORT(A, p, r)
1.  if p < r
2.      q = ⌊(p + r)/2⌋
3.      MERGE-SORT(A, p, q)
4.      MERGE-SORT(A, q+1, r)
5.      MERGE(A, p, q, r)
```

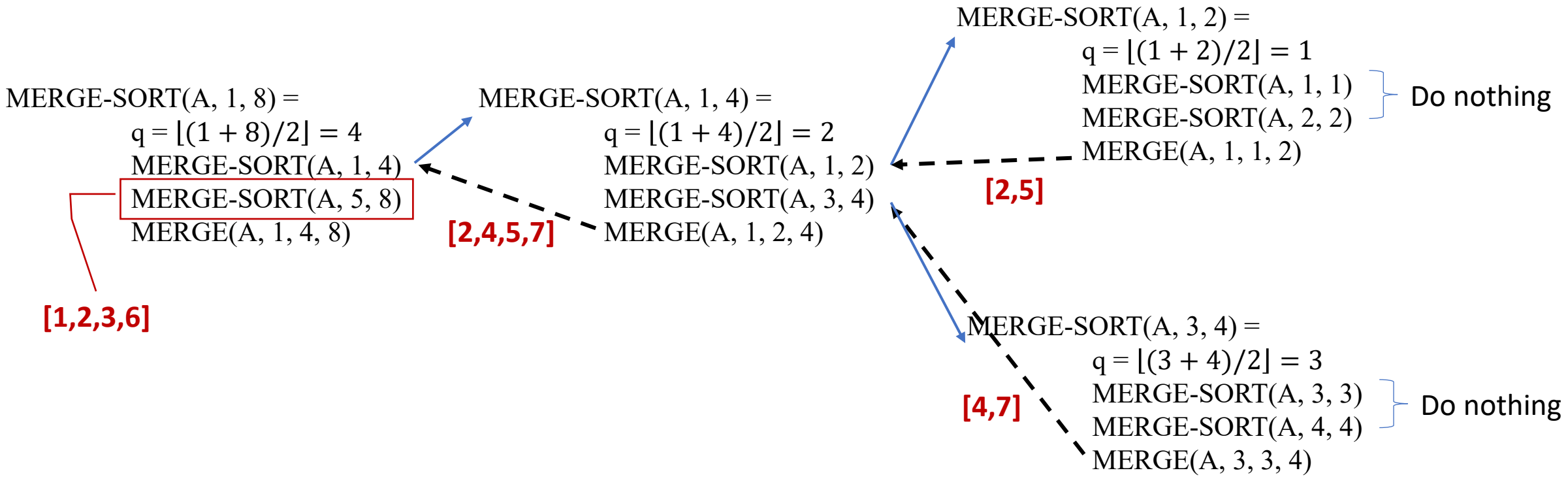
Divide

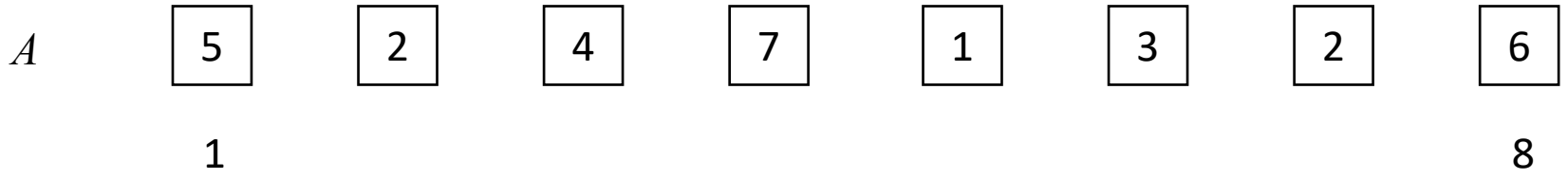
Conquer

Combine

- It sorts the elements in the subarray $A[p:r]$
- If $p == r$, do nothing because A has one element and is therefore already sorted
- Otherwise, divide A into two subarrays:
  - $A[p:q]$, containing $\lceil n/2 \rceil$ items
  - $A[q+1:r]$, containing $\lfloor n/2 \rfloor$ items
- Call MERGE-SORT on each subarray
- Call MERGE

Visualizing merge sort





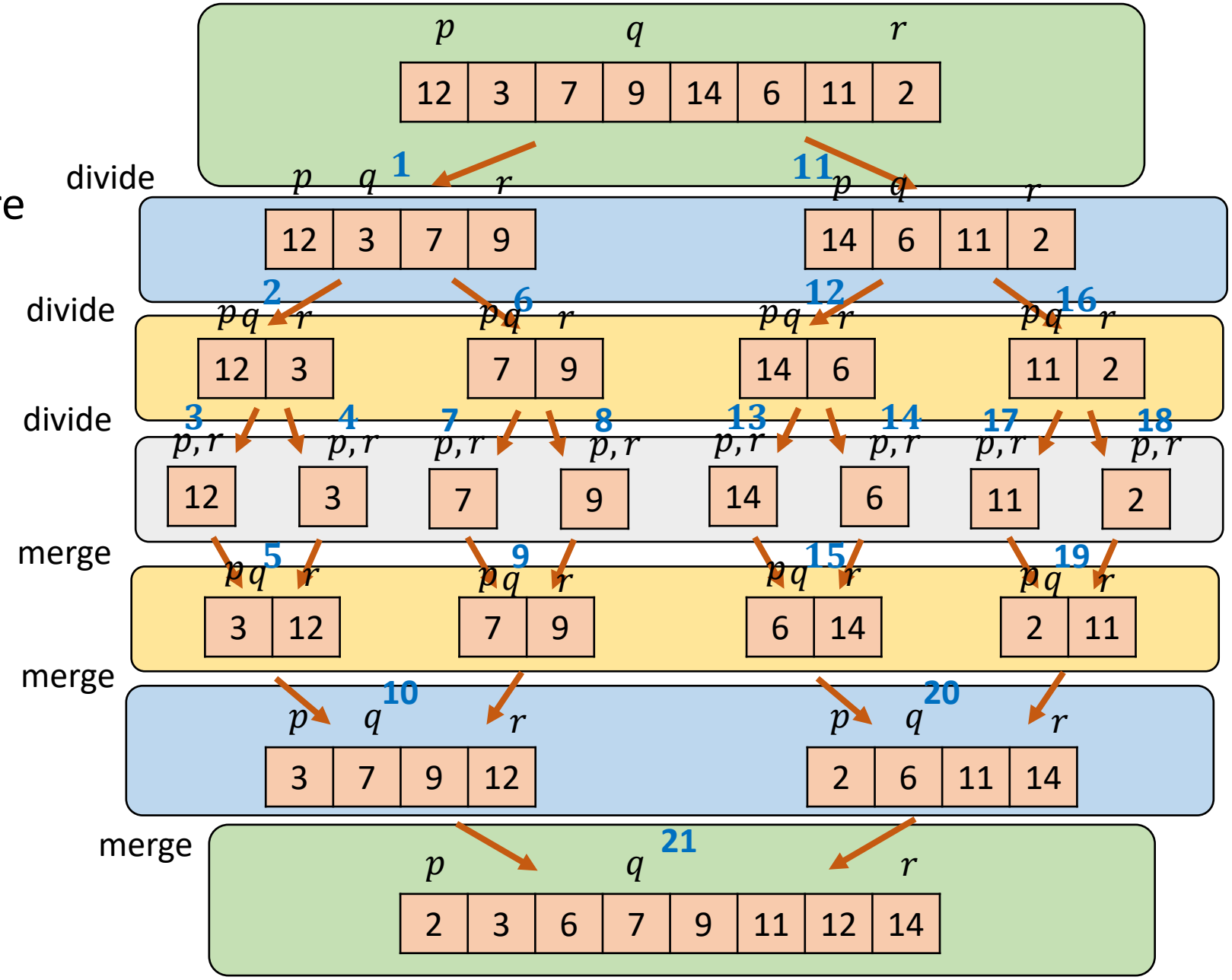
Example

- Merge-sort procedure

```
MERGE-SORT(A, p, r)
1.   if p < r
2.        q = ⌊(p + r)/2⌋
3.        MERGE-SORT(A, p, q)
4.        MERGE-SORT(A, q+1, r)
5.        MERGE(A, p, q, r)
```

```
MERGE(A, p, q, r)
.....

12.  for k = p to r
13.       if L[i] ≤ R[j]
14.            A[k] = L[i]
15.            i = i + 1
16.       else A[k] = R[j]
17.            j = j + 1
```



Running time of merge sort

- ❑ MERGE-SORT contains a recursive call to itself
- ❑ We can describe its running time by using a ***recurrence equation***:
  - Which describes the overall running time on a problem of size n
  - Then solve the recurrence equation with mathematical tools

- ❑ Let $T(n)$ be the running time for a problem of size n
  - If the problem size is smaller enough, say $n \leq c$, for some constant c, then the straightforward solution takes constant time, $\Theta(1)$.
  - Suppose we divide the problem into a subproblems, each of which is $\mathbf{1/b}$ the size of the original problem
    - ✓ For merge sort, $a = b = 2$, but sometimes $a \neq b$
  - It takes $\boldsymbol{T(n/b)}$ time to solve one subproblem, and so it takes $\boldsymbol{aT(n/b)}$ to solve a of them

Running time of merge sort (cont.)

- Suppose that it takes $\boldsymbol{D(n)}$ time to divide into subproblems and $\boldsymbol{C(n)}$ time to combine the solutions to subproblems into the solution to the original problem.
- ***Recurrence equation***

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n), & \text{otherwise} \end{cases}$$

- This equation can be solved.
- **Divide-and-conquer** algorithms yield such form of running time

Analysis of merge sort

- Running time of merge-sort : T(n)

```
MERGE-SORT(A, p, r)
1.   if p < r
2.        q = ⌊(p + r)/2⌋
3.        MERGE-SORT(A, p, q)
4.        MERGE-SORT(A, q+1, r)
5.        MERGE(A, p, q, r)
```

```
MERGE(A, p, q, r)
……

12.  for k = p to r
13.       if L[i] ≤ R[j]
14.            A[k] = L[i]
15.            i = i + 1
16.       else A[k] = R[j]
17.            j = j + 1
```

- ✓ **Divide**: just computes the middle of the array, which takes constant time. Thus, $D(n) = \Theta(1)$.
- ✓ **Conquer**: recursively solve two subproblems, each of size $n/2$, which contributes $2T(n/2)$ to the running time.
- ✓ **Combine**: we already know that the MERGE procedure on a subarray of size n takes $\Theta(n)$ time, and so $C(n) = \Theta(n)$.

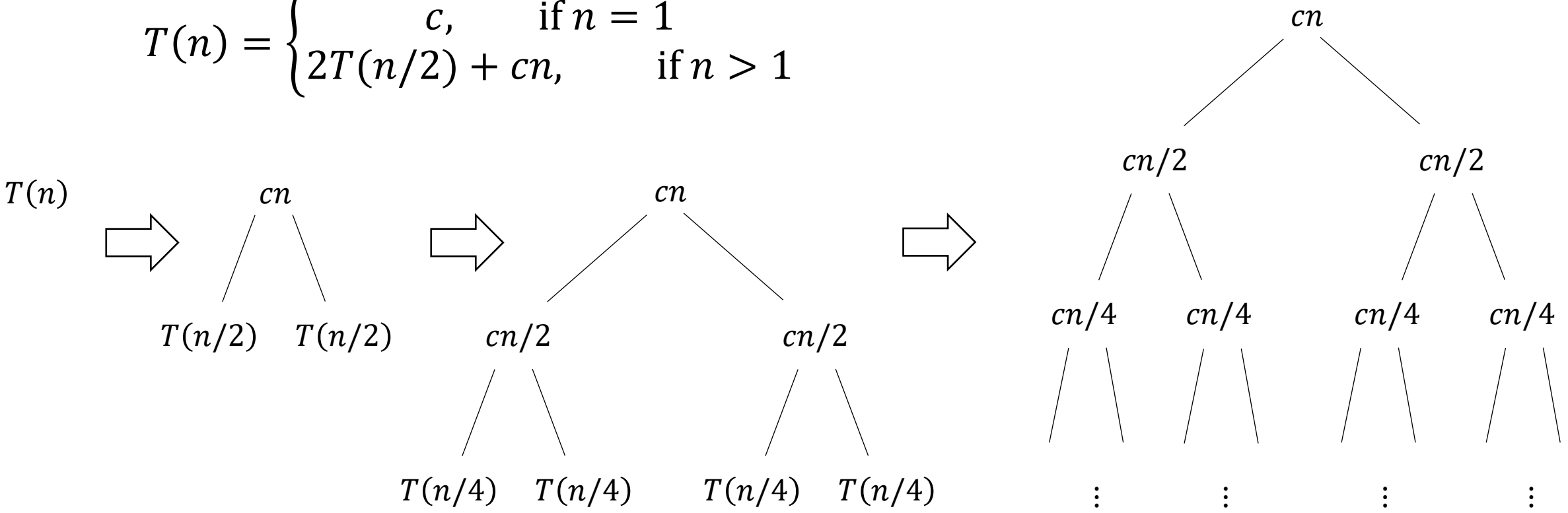
$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) + \Theta(1), & \text{if } n > 1 \end{cases}$$

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 2T(n/2) + \Theta(n), & \text{if } n > 1 \end{cases}$$

Intuitive understanding of $T(n)$

- Rewrite recurrence equation

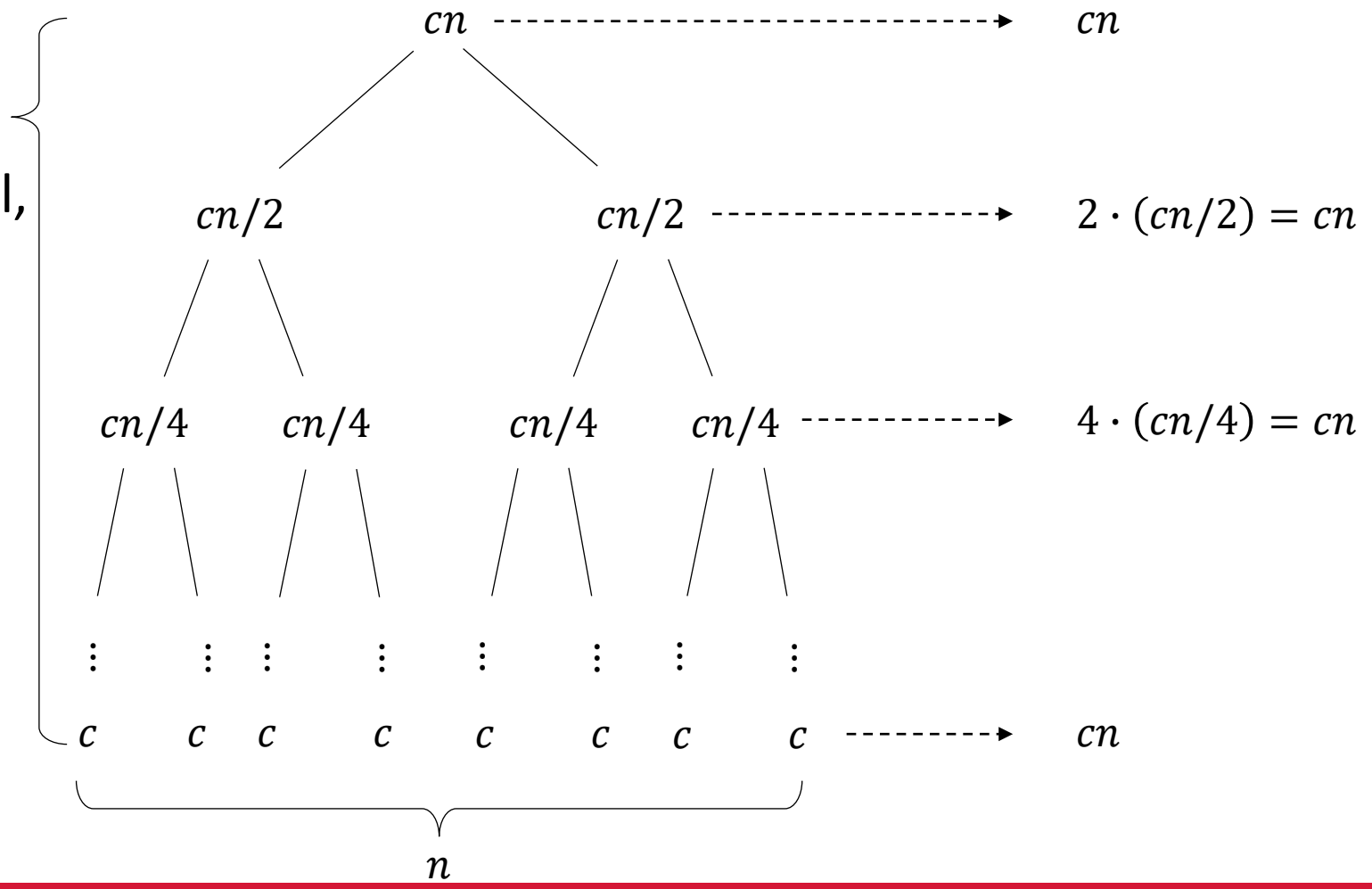
$$T(n) = \begin{cases} c, & \text{if } n = 1 \\ 2T(n/2) + cn, & \text{if } n > 1 \end{cases}$$



Intuitive understanding of $T(n)$

- For level i,
  the number in the node is $c\frac{n}{2^i}$
- When it reaches the lowest level,
$c\frac{n}{2^i} = c$,i.e., $\frac{n}{2^i} = 1$ ➔ $i = \lg n$
- Number of levels = $\lg n + 1$

- Total running time =
  $(\lg n + 1) \cdot cn = cn \lg n + cn$
- Ignoring the low-order term:
  $\boldsymbol{\Theta(n \lg n)}$



Comparing with insertion sort

- Insertion sort: worst-case running time is $\boldsymbol{T(n) = \Theta(n^2)}$
- Merge sort: worst-case running time is $\boldsymbol{T(n) = \Theta(n \lg n)}$
- n^2 grows faster than $n \lg n$, and thus merge sort runs faster (when n is large enough)

Example

Problem

- Find the exact total cost (running time) of the following recurrence equation using recursion tree.
  - Let's assume that n is the exact power of 2.

$$T(n) = \begin{cases} 1 & ,n = 1 \\ T\left(\frac{n}{2}\right) + n^3 & ,n > 1 \end{cases}$$

Q & A